MINI PROJECT REPORT

Text Cleaner and Keyword Finder Using NLP

Subject	Natural Language Processing (NLP)
Environment	Python in Google Colab
Dataset	150 sample news headlines and short messages
Core techniques	Text cleaning, stop-word removal, stemming, word frequency, TF-IDF

Student name: Apurwa

Roll number: 6

Under the guidance of: Mrudul Ma'am

College: ISBM College of Engineering, Nande, Pune

Department: Artificial Intelligence and Data Science

Academic year: 2026–2027

1. Abstract

This project demonstrates a basic Natural Language Processing workflow for preparing short text documents and extracting useful information from them. A dataset of 150 sample news headlines and short messages is created and saved as a CSV file. The text is converted to lowercase, URLs and non-letter characters are removed, English stop words are filtered out, and Porter stemming is applied. The project then visualizes the ten most frequent cleaned word stems and ranks terms by their average TF-IDF score across the collection.

2. Aim and Objectives

Aim: To clean a collection of short texts and identify frequent words and informative keywords using NLP techniques.

Objectives

• Create and load a CSV dataset containing 150 short texts.
• Convert text to lowercase and remove URLs, punctuation, numbers, and special characters.
• Remove English stop words and apply Porter stemming.
• Count word frequencies and display the top ten terms in a bar chart.
• Use TF-IDF to calculate average term scores and report the top ten terms.
• Export the original and cleaned text to a CSV file.

3. Tools and Libraries

Tool / library	Purpose
Google Colab	Cloud notebook environment for running Python code.
Pandas	Creating, loading, processing, and exporting tabular data.
NLTK	English stop-word list and Porter stemming.
Regular expressions (re)	Removing URLs and unwanted characters from text.
Collections.Counter	Counting how often each cleaned word occurs.
Matplotlib	Plotting the top-ten word-frequency bar chart.
Scikit-learn	Calculating TF-IDF features using TfidfVectorizer.

4. Methodology

Step 1 — Dataset creation and loading

A list of 150 sample headlines and short messages is stored in a Pandas DataFrame with one column named `text`. The DataFrame is exported as `sample_texts.csv` and then loaded for processing.

Step 2 — Text preprocessing

Each text is converted to lowercase. Regular expressions remove URLs and all characters other than lowercase English letters and whitespace. The text is split into tokens; English stop words are excluded, and Porter stemming is applied to the remaining words. The cleaned tokens are joined back into a string and stored in a new `cleaned_text` column.

Step 3 — Word-frequency analysis

All cleaned texts are combined into a single sequence of tokens. `Counter` counts the occurrences of each token, and the ten most common terms are plotted using Matplotlib.

Step 4 — TF-IDF keyword extraction

Scikit-learn's `TfidfVectorizer` converts the cleaned documents into a TF-IDF matrix. The mean TF-IDF score of each term across all documents is calculated, and the ten terms with the highest average scores are displayed. These are collection-level rankings rather than keywords for a single document.

Step 5 — Export

The original and cleaned text columns are saved to `cleaned_texts.csv` so that the preprocessing output can be inspected separately from the notebook.

5. Dataset Output

The executed notebook confirms that the dataset contains 150 texts. The screenshot below shows the dataset creation cell and preview of the first ten rows.

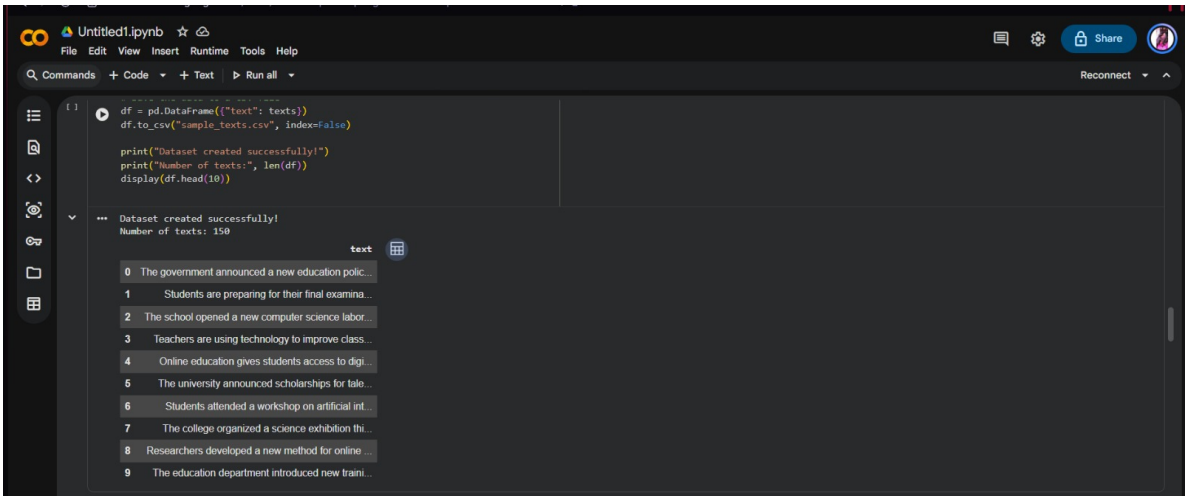


Figure 1. Creation of the 150-text sample dataset and preview of its first rows.

6. Text Cleaning Output

The output shows the original text alongside its cleaned version. Common words have been removed and the remaining words have been stemmed. Porter stemming can produce shortened forms that are not complete dictionary words; this is expected behaviour.

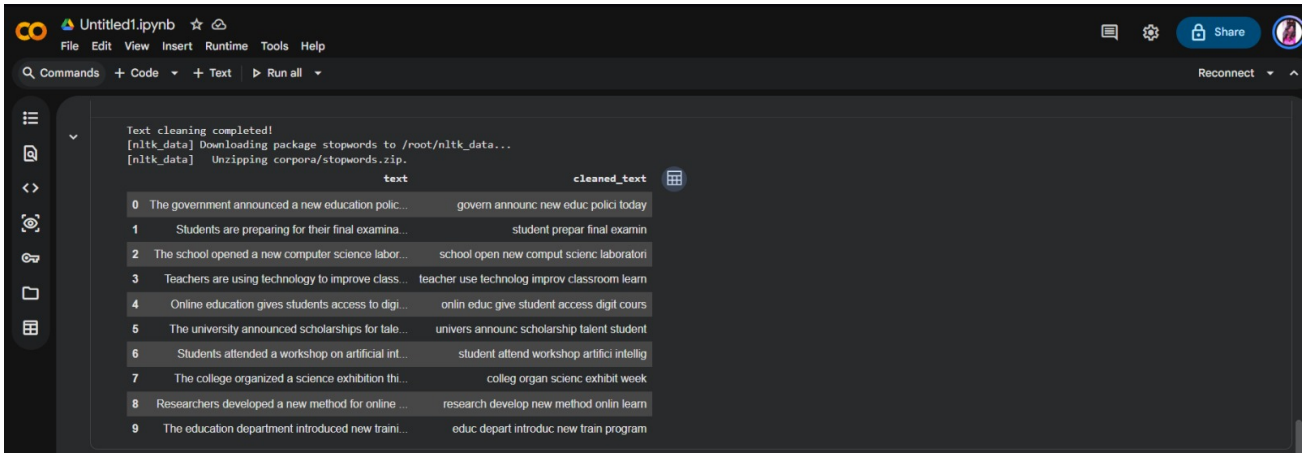


Figure 2. Original text and cleaned text shown in the Colab notebook.

7. Top 10 Most Frequent Words

The chart generated by the executed code displays the ten most frequent terms. Counts below are transcribed from the supplied output screenshot.

Rank	Cleaned term	Frequency
1	new	25
2	research	13
3	announc	12
4	improv	11
5	develop	10
6	student	8
7	introduc	8
8	onlin	7
9	team	7
10	scientist	7

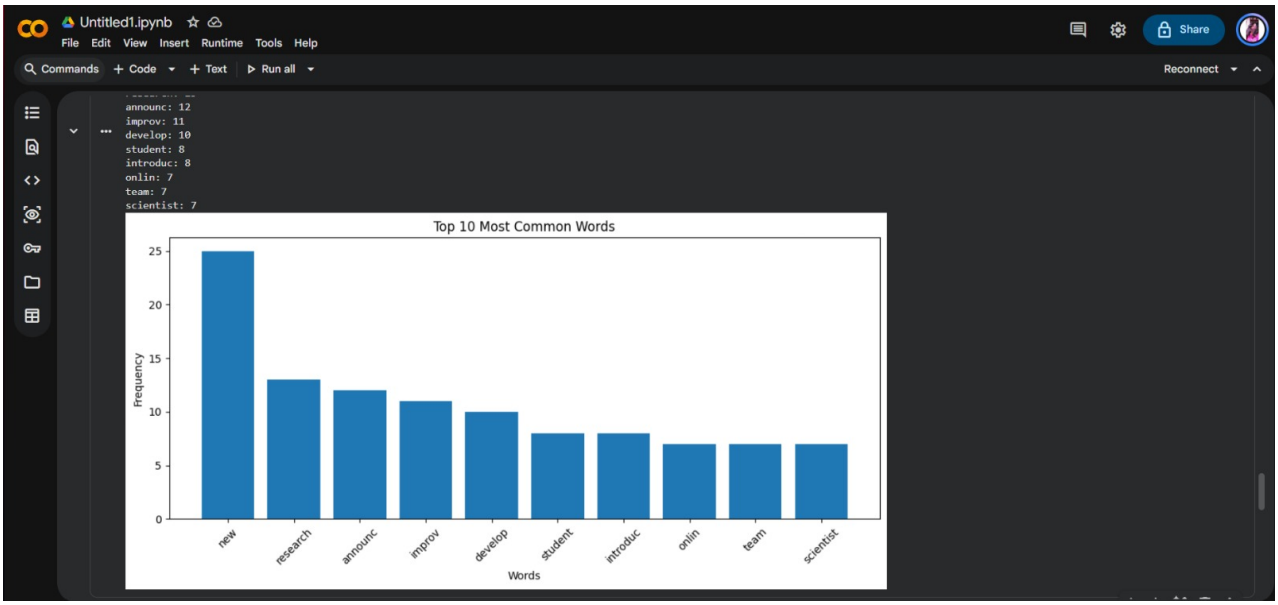


Figure 3. Bar chart of the ten most frequent cleaned word stems.

8. Top 10 TF-IDF Keywords

The notebook ranks terms by their average TF-IDF score across the full dataset. Values below are transcribed from the supplied Colab screenshot.

Rank	Term	Average TF-IDF score
1	new	0.0437
2	research	0.0284
3	announc	0.0271
4	improv	0.0236
5	develop	0.0233
6	student	0.0200
7	introduc	0.0189
8	team	0.0180
9	onlin	0.0172
10	farmer	0.0167

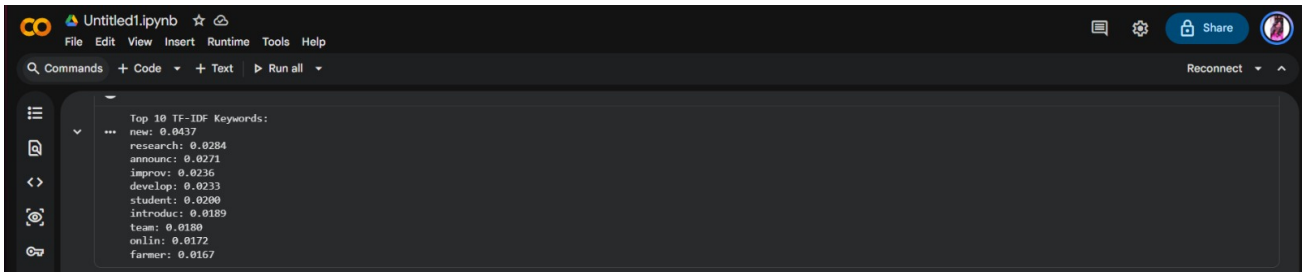


Figure 4. TF-IDF keyword output from the executed notebook.

9. Discussion

The word-frequency chart shows which stems occur most often in the collection; “new” is the most frequent term in the observed output. TF-IDF provides a different ranking by considering term frequency within documents and how widely a term appears across the collection. The results reflect this particular sample dataset and should not be generalized to all news or SMS text. TF-IDF scores are relative weights, not probabilities.

10. Conclusion

The project demonstrates a basic NLP pipeline in Python: creating and loading a text dataset, cleaning text, removing stop words, applying stemming, counting word frequencies, visualizing frequent terms, and extracting keywords using TF-IDF. The processed data is exported to CSV, making the results easy to inspect and share on GitHub.

11. Files Included in the GitHub Repository

- `NLP_Text_Cleaner.ipynb` — executed notebook containing code and outputs.
- `sample_texts.csv` — original dataset of 150 texts.
- `cleaned_texts.csv` — original and cleaned text columns.
- `NLP_Text_Cleaner_Keyword_Finder_Report.pdf` — this report with output screenshots.

Output values and screenshots are based on the Colab screenshots supplied for this report.